

Timing diagram for execution block:

1. The `execute\_top` module acts as the centralized execution stage of the accelerator pipeline, interfacing fetch/decode outputs with arithmetic, neural-network, and matrix-computation hardware blocks through opcode-driven control.
 2. The architecture integrates multiple AI-oriented compute units including MAC, dot-product, activation functions (ReLU, Sigmoid, Tanh), BNN operations, scaling, clipping, and ArgMax within a unified parallel execution framework.
 3. Dedicated hardware accelerators for conventional 4×4 matrix multiplication and systolic-array-based multiplication are selectively enabled using control signals (`valid\_mat`, `en\_sys`) to support heterogeneous computation modes.
 4. A hierarchical timing scheme is implemented using `clk` for internal accelerator operation and a slower `bigclk` for pipeline-stage synchronization, enabling controlled execution latency and simplified inter-stage data transfer.
 5. The design supports pipelined processor integration by forwarding register addresses and control signals across stages, allowing seamless connection between fetch/decode, execute, and writeback modules in a modular AI accelerator datapath.
- * The accelerator uses a dual-clock timing scheme where `clk` controls internal computations and `bigclk` synchronizes pipeline-stage execution.
 - * Input matrices are diagonally skewed through `a\_pad` and `b\_pad` registers to ensure synchronized operand arrival across the systolic array.
 - * During each clock cycle, operands propagate horizontally and vertically between Processing Elements (PEs) using `a\_wire` and `b\_wire` interconnects.
 - * Partial sums are accumulated recursively through the `accumulator → acc\_next` feedback path, enabling pipelined MAC computation.
 - * Control signals such as `valid\_mat`, `en\_sys`, `done\_mat`, and `done\_sys` manage accelerator enable, pipeline flushing, and output-valid timing before writeback.

mat_A=[0 1 2 3

1 2 3 4

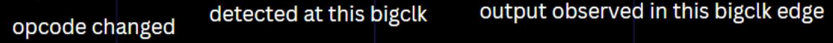
2 3 4 5

5 6 7 8]

Mat_B= [1 2 3 4

5 6 7 8

13 14 15 16]



GTKWave - waveform.vcd

File Edit Search Time Markers View Help

From: 0 sec To: 1400 ns Marker: -- Cursor: 651800 ps

SST

- execute_top_tb
 - dot_u
 - bnn_u
 - clip_u
 - dot_u
 - mac_u
 - mat
 - max_u
 - pop_u
 - relu_u
 - scale_u
 - sig_u

Signals

Type	Signals
wire	addr_a[3:0]
wire	addr_a_next[3:0]
wire	addr_b[3:0]
wire	addr_b_next[3:0]
wire	bigclk
wire	bnn_out[31:0]
wire	check1
wire	check2
wire	check3
wire	check4
wire	clip_out[31:0]
wire	clk
reg	cout[31:0]
wire	done_mat
wire	done_vec

Filter:

Append Insert Replace

Waves

500 ns 600 ns 700 ns 800 ns

clk

bigclk

reg_a[7:0] 0A

reg_b[7:0] 02

rst

opcode[3:0]

cout[31:0]

bnn_out[31:0] 00000005 00000014 000000A8 0000000A 00000000

clip_out[31:0] 00000003 00000007 0000000A 0000000A 00000000

dot_prod_out[31:0] FFFFFFF0 00000014 00000000 00000000 00000000

mac_out[31:0] FFFFFFF6 00000000 00000000 00000000 00000000

max_index[2:0] 011

max_value[31:0] 00000004

mult_result[31:0] FFFFFFF0 00000014 00000000 00000000 00000000

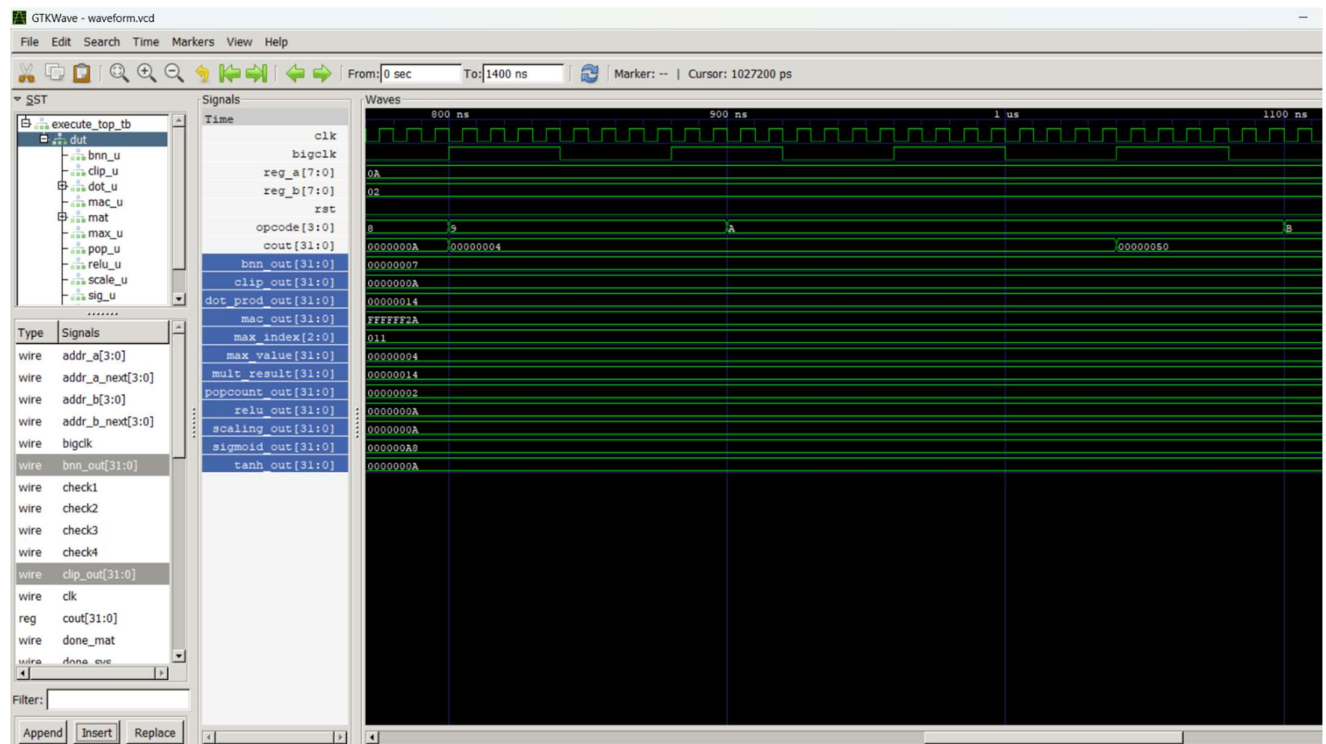
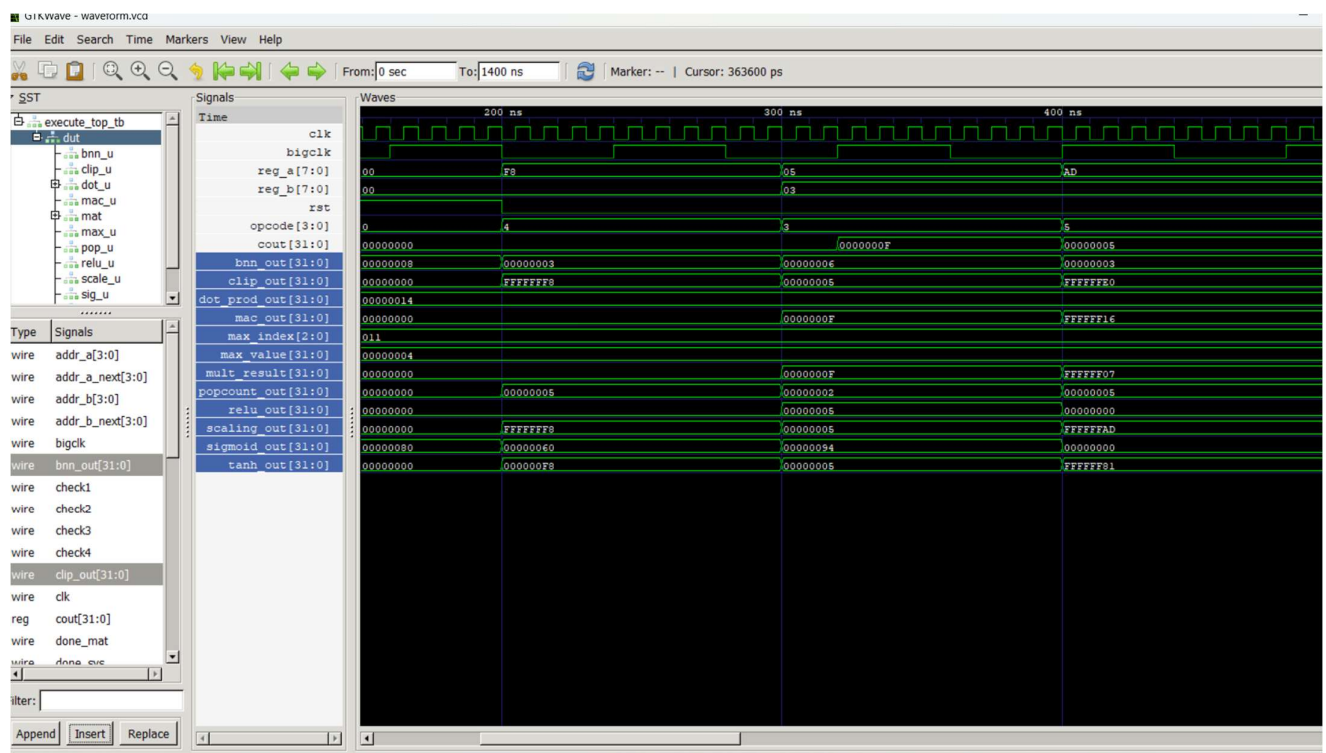
popcount_out[31:0] 00000005 00000002 00000000 00000000 00000000

relu_out[31:0] 00000000 0000000A 0000000A 0000000A 00000000

scaling_out[31:0] FFFFFFFAD 00000000 00000000 00000000 00000000

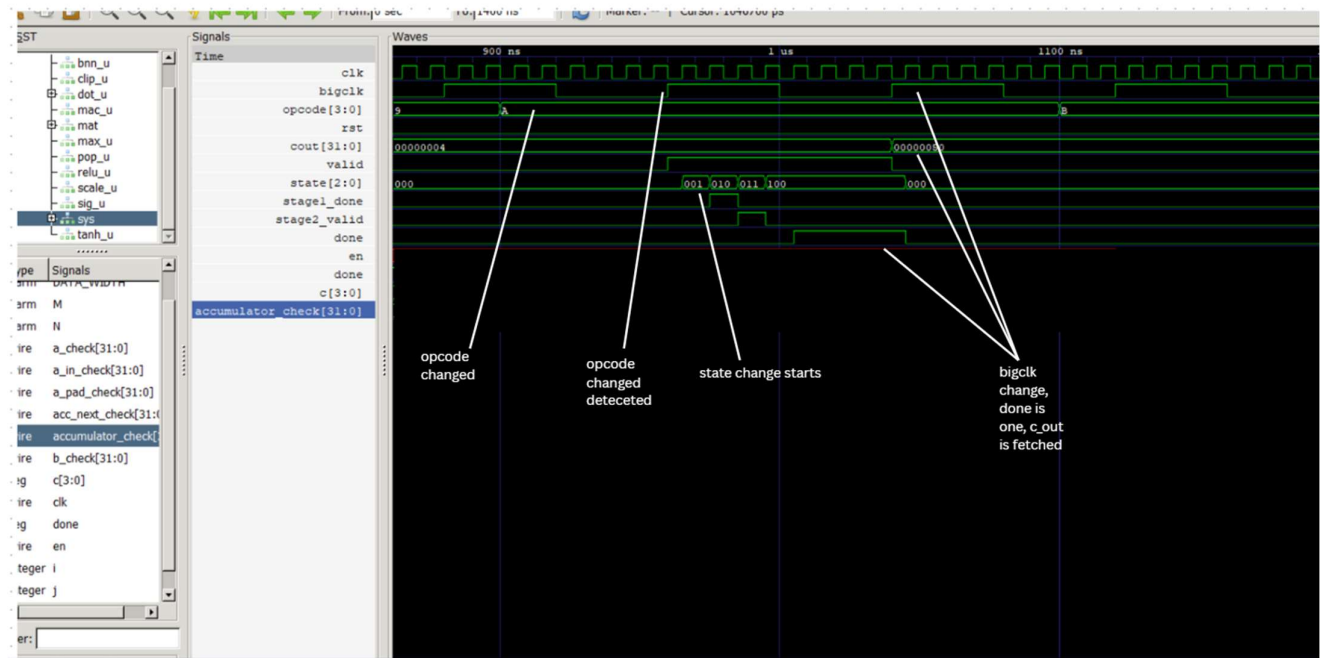
sigmoid_out[31:0] 00000000 000000A8 00000000 00000000 00000000

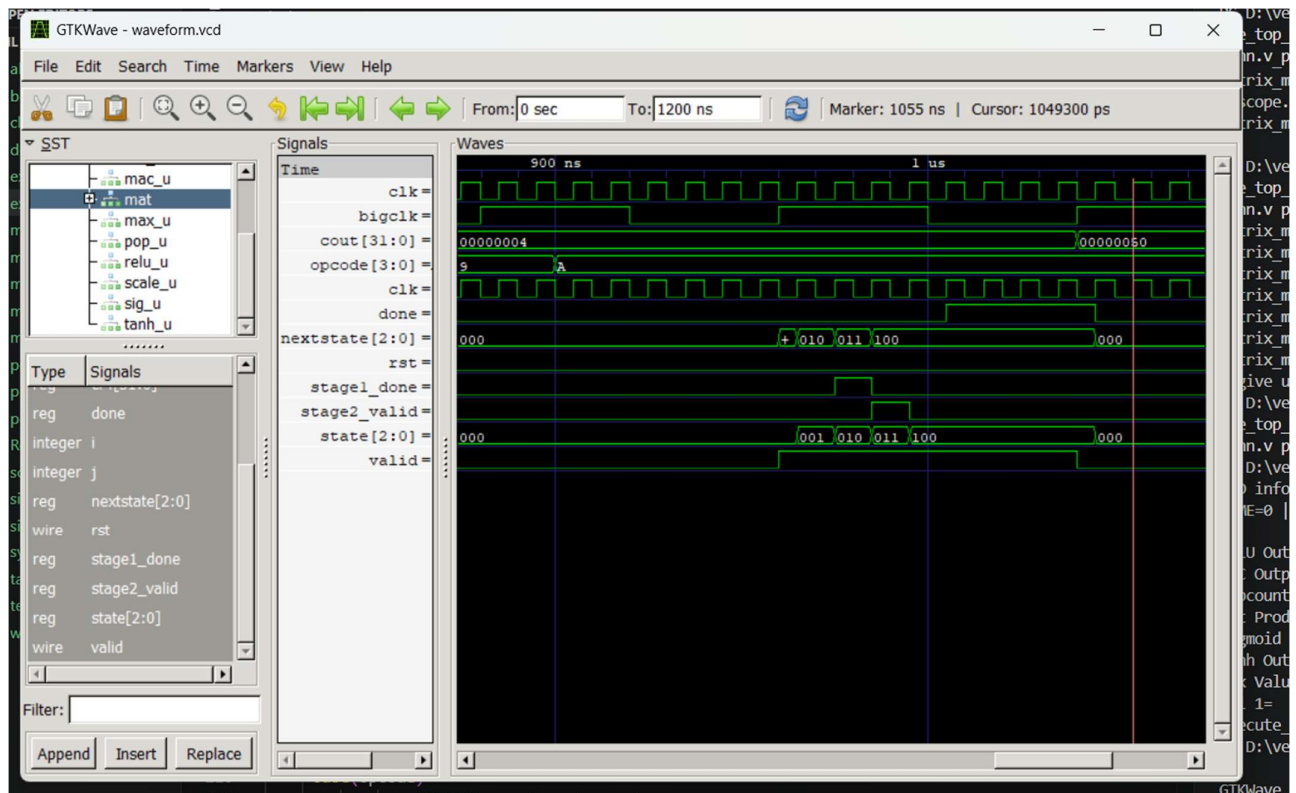
tanh_out[31:0] FFFFFFFB1 00000000 00000000 00000000 00000000



For matrix_mult:

- The matrix_mult_4x4 module implements a parallel 4x4 matrix multiplication architecture using 16 concurrent dot_product_4 computation units for high throughput operation.
- An FSM-based control path manages data loading, transpose generation, computation triggering, result storage, and completion signaling across multiple pipeline stages.
- Matrix B is internally transposed to improve row-column data access efficiency, enabling simultaneous dot-product evaluation for all output elements.
- The design uses staged valid-control signals (stage1_done, stage2_valid, done) to synchronize pipeline execution and ensure deterministic timing behavior during hardware acceleration.





Systolic :

The accelerator implements a diagonally skewed data-injection scheme using padded shift registers (a_pad and b_pad) to guarantee cycle-accurate operand alignment across the systolic fabric without requiring complex control routing. The accelerator uses a scalable 8×8 systolic-array architecture with diagonally skewed data injection, pipelined MAC-based distributed accumulation, and operand forwarding for temporal data reuse, while a lightweight counter-driven timing scheme handles synchronization and result generation without centralized FSM control. Matrix tilling id done by processor

A lightweight cycle-driven control mechanism using the internal counter c manages operand streaming, synchronization, pipeline flushing, and result-valid timing, eliminating the need for a centralized FSM-based scheduler.

